



System Health Monitor Overview

Overview & Implementation Steps

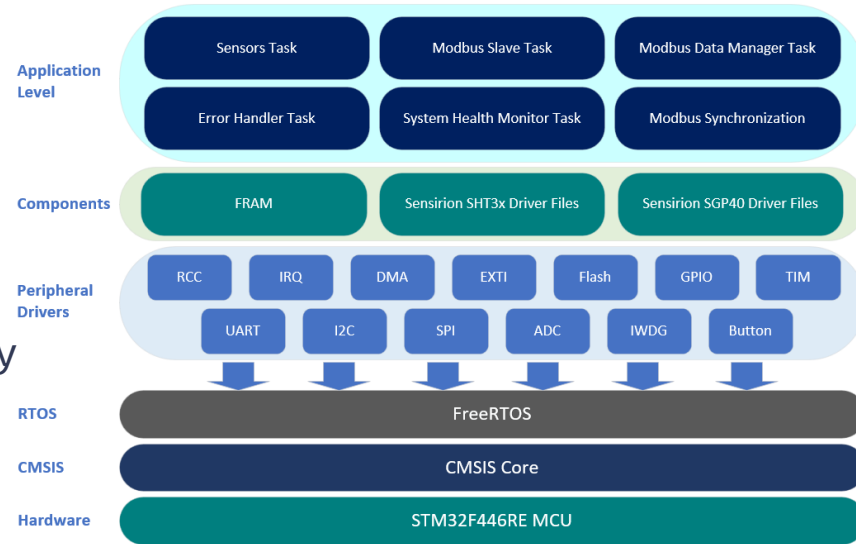
System Health Monitor Overview

- System Health Monitor Overview
- Section Implementation Steps

System Health Monitor Overview Part I

General

- **Purpose & Functionality:** Monitors MCU's internal temperature to ensure safe operation and resets the Independent Watchdog Timer (IWDG) if all critical tasks are verified as running.
- **Project Significance:**
 - **Safety Monitoring:** Enhances reliability by monitoring temperature anomalies.
 - **Watchdog Management:** Ensures system responsiveness by verifying that critical tasks are operational.
 - **User Interaction:** Allows users to acknowledge previous watchdog resets upon startup.

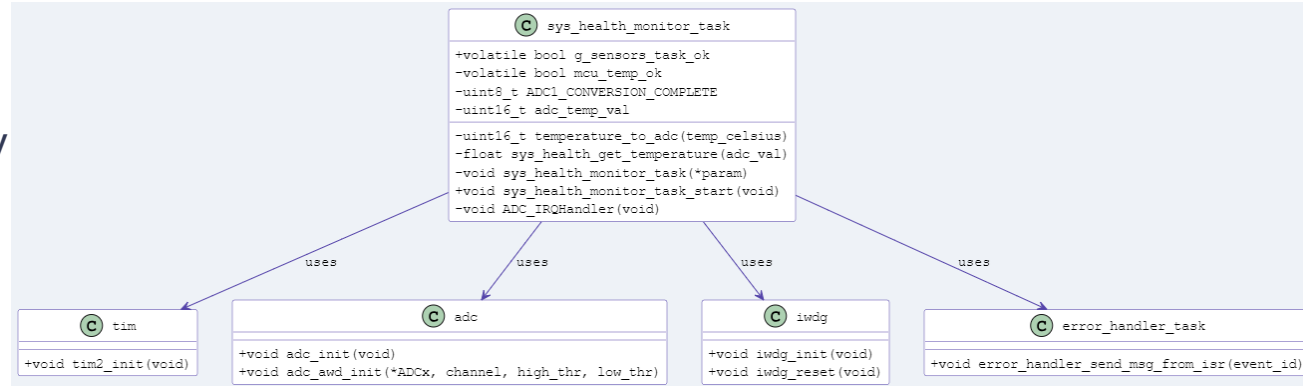


System Health Monitor Task Overview Part II

System Health Monitor Task: Monitors MCU's internal temperature, checks critical task status, resets the watchdog timer, and communicates with the error handler.

- **FreeRTOS Task:** Loop with a periodic delay, checks critical tasks and resets IWDG.
- **Temperature Monitoring:** ADC setup on a timer trigger to obtain internal MCU temperature and the ADC's Analog Watchdog (AWD) monitors pre-defined thresholds.
- **Interrupt Handling:** **ADC_IRQHandler** handles ADC conversion completion, Analog Watchdog monitoring, and communicates with the error handler task.

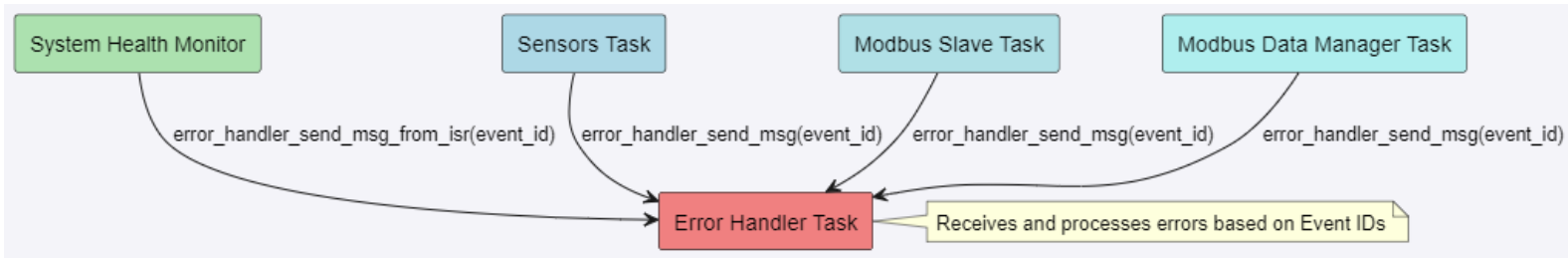
- **Project Significance:**
The task's design allows for extensibility and serves as a basic example for System Health Monitoring.



System Health Monitor Task Overview Part III

System Health Monitor Temperature Alarms: Communication facilitated through `error_handler_send_msg_from_isr`, via the `ADC_IRQHandler` Analog Watchdog Interrupts.

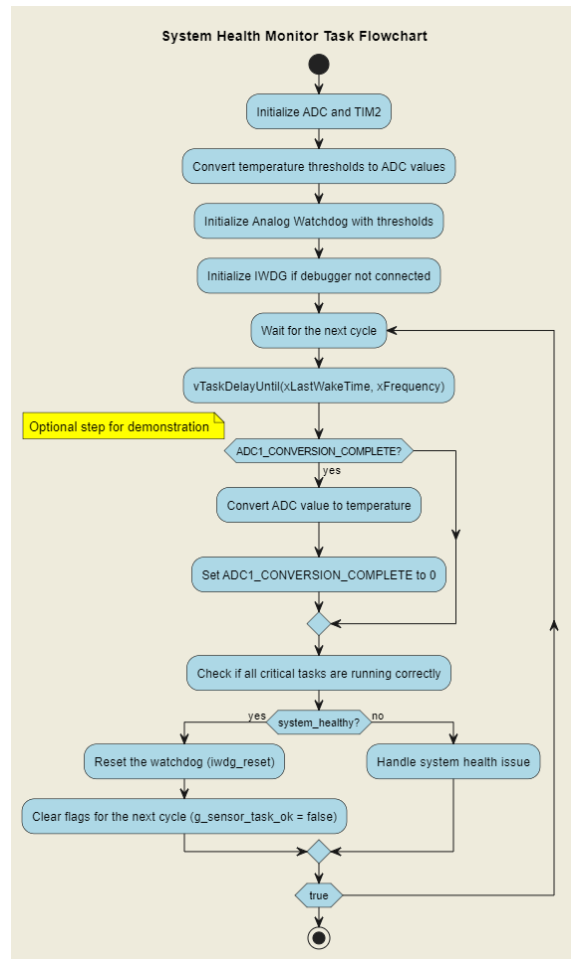
- **Analog Watchdog:** Monitors converted analog values against high and low thresholds. Triggers an interrupt if values fall outside these limits, detecting abnormal temperature conditions.
- **Communication Mechanism:** `ADC_IRQHandler` calls `error_handler_send_msg_from_isr` with `EVT_SYS_HEALTH_AWDG_THRESHOLD_EXCEEDED` event ID to notify the Error Handler Task.
- **Project Significance:** Ensures the System Health Monitor has immediate awareness of temperature threshold violations and communicates alarms to the Error Handler.



System Health Monitor Task Overview Part IV

Task Overview: Explanation related to the flow diagram.

- **Timer-Triggered ADC:** Transfers the responsibility of getting ADC values and checking the MCU temperature to the [ADC_IRQHandler](#).
- **AWD Initialization:** Allows for customization of the Analog Watchdog's upper and lower thresholds.
- **IWDG Initialization:** Initialized only if the debugger is not connected, ensuring the watchdog is enabled only when necessary.
- **Periodic Critical Task Checks:** Verifies that critical tasks are operational and that the internal MCU temperature is within the acceptable range before resetting the watchdog.
- **Project Significance:** A basic System Health Monitor for demonstration purposes, which can be expanded for specific application needs.



System Health Monitor Files Preparation

Overview

- **Objective:** Prepare template .c/.h files for integrating additional components as they are developed, e.g., ADC, IRQ, TIM, IWDG, EXTI & Button (to be used at Startup).
- **Key Steps:**
 - Create `sys_health_monitor.h` with Analog Watchdog thresholds, critical task flag, and function prototype.
 - Create `sys_health_monitor.c` with health status flags, ADC conversion completion flag, ADC temperature variable and task functions.
- **Project Significance:** Sets up the System Health Monitor Task for ongoing development and integration.

 `sys_health_monitor_task`

```
+volatile bool g_sensors_task_ok
-volatile bool mcu_temp_ok
-uint8_t ADC1_CONVERSION_COMPLETE
-uint16_t adc_temp_val

-uint16_t temperature_to_adc(temp_celsius)
-float sys_health_get_temperature(adc_val)
-void sys_health_monitor_task(*param)
+void sys_health_monitor_task_start(void)
-void ADC_IRQHandler(void)
```

ADC, TIM & IRQ Priority Subsection

Overview

- **Objective:** Provide an overview and implement ADC, TIM, and IRQ drivers required for the System Health Monitor.
- **Key Steps:**
 - Review the ADC & TIM trigger settings needed for obtaining the internal MCU temperature and IRQ priority requirements for FreeRTOS.
 - Implement ADC initialization (`adc.c/.h` files).
 - Implement TIM trigger for ADC conversions (`tim.c/.h` files).
 - Define IRQ priorities and implement the ``set priorities`` function (`irq.c/.h` files).
- **Project Significance:**
 - This development section focuses on achieving the internal MCU temperature monitoring for the System Health Monitor.

System Health Monitor Temperature Implementation & Testing

Overview

- **Objective:** Integrate IRQ set priorities, ADC & TIM initialization functions, implement the ADC IRQ Handler, Temperature Conversion, and Get Temperature functions.
- **Key Steps:**
 - **Define Functions:** ``temperature_to_adc`` and ``get_temperature``.
 - **Integration:** Call IRQ Set Priorities, and TIM and ADC Initialization functions.
 - **Define ADC IRQ Handler:** Check if the Analog Watchdog and End of Conversion flags are set and take the appropriate actions.
 - **Testing:** Run the System Health Monitor task and verify ADC interrupts, Internal MCU temperature, and Analog Watchdog interrupts.
- **Project Significance:** Confirms that the Internal MCU Monitoring portion of the System Health Monitor is operational and prepares for the next steps.

IWDG, EXTI & Button Sub-Section

Overview

- **Objective:** Provide an overview and implement the IWDG, EXTI, and Button drivers required for the System Health Monitor and user acknowledgment of IWDG resets.
- **Key Steps:**
 - Review the IWDG configuration, EXTI driver and Button implementation.
 - IWDG implementation (`iwdg.c/.h` files).
 - EXTI implementation (`exti.c/.h` files).
 - Button implementation (`button.c/.h` files).
- **Project Significance:**
 - Focuses on implementing the Independent Watchdog for the System Health Monitor and user-button acknowledgment for previous IWDG resets at startup.

IWDG/User-Button Acknowledgement Implementation & Testing

Overview

- **Objective:** Integrate Button initialization and check/acknowledge functions at Startup, and IWDG reset into the System Health Monitor, and perform testing.
- **Key Steps:**
 - **Integration:** Call ``button_init`` & ``button_check_and_acknowledge_iwdg_event`` at Startup and ``iwdg_init`` & ``iwdg_reset`` within the System Health Monitor task.
 - **Testing:** Run the project to cause an Independent Watchdog Reset and test USER-button acknowledgement.
- **Project Significance:** Confirms that the Independent Watchdog portion of the System Health Monitor is operational and that USER-button acknowledgements allow for acknowledging and clearing the event.

The background features a complex network of thin, light-colored lines and small circles, forming various triangular shapes. Some triangles are filled with a light yellow or green color, while others are outlined. The overall effect is a subtle, geometric pattern that suggests a network or a system.

Next: System Health Monitor Files Preparation
